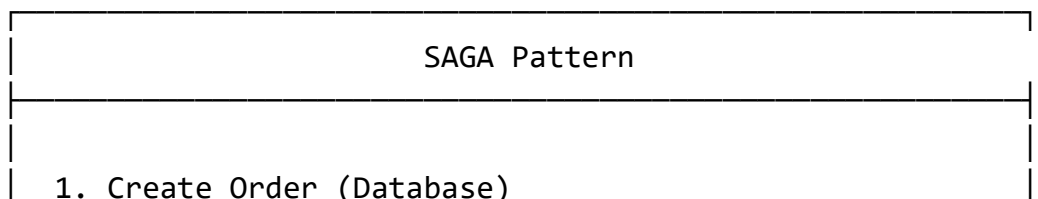
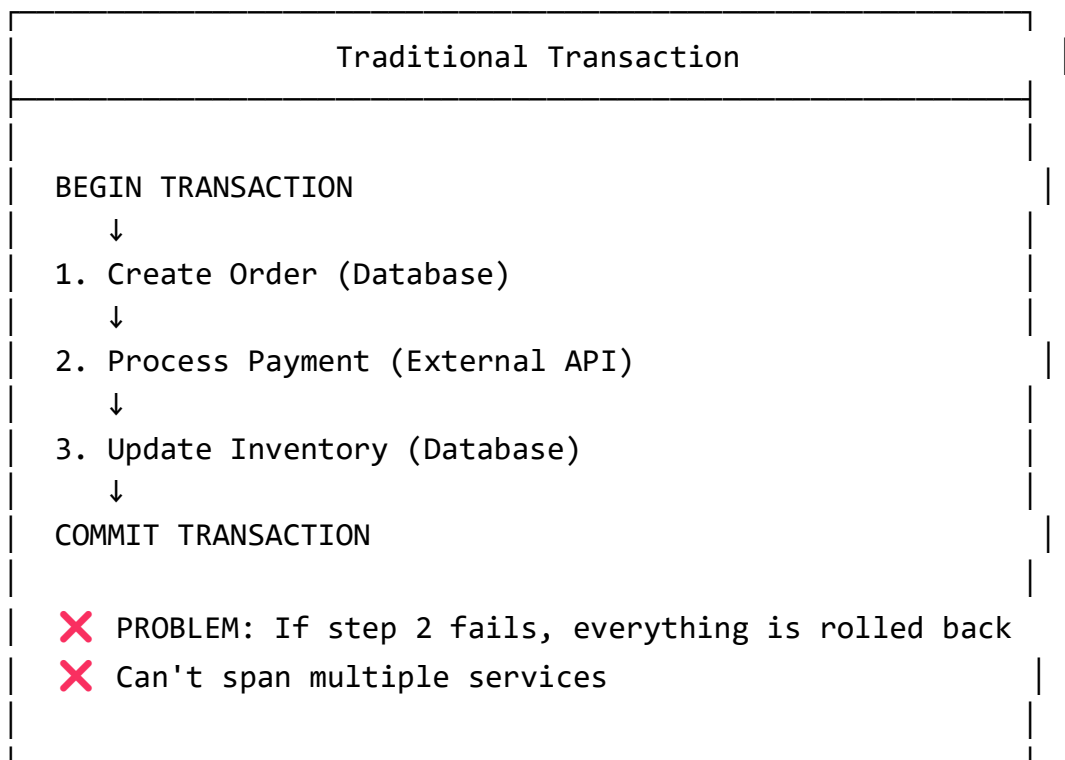


SAGA Pattern - Complete Guide Using Your Project

Part 1: What is SAGA Pattern?

Definition

SAGA pattern is a **sequence of local transactions** where each transaction performs an action and publishes an event. If any transaction fails, the saga executes **compensating transactions** to undo the previous actions.



- ↓ Publish OrderCreatedEvent
- 2. Process Payment (External API)
 - ↓ Publish PaymentCompletedEvent
- 3. Update Inventory (Database)
 - ↓ Publish InventoryUpdatedEvent

If any step fails:

- Payment failed → Publish PaymentFailedEvent
- Order service compensates → Cancel Order
- Inventory compensates → Restore Inventory

Why SAGA is Needed

The Problem: Distributed transactions across microservices

ECommerce Order Flow

User clicks "Checkout"

↓

Basket Service → Delete Basket

↓

Ordering Service → Create Order

↓

Payment Service → Process Payment

↓

Inventory Service → Update Stock

- ✗ What if Payment Service fails?
- ✗ Order created but payment failed
- ✗ Basket deleted but order failed
- ✗ Inconsistent state!

SAGA Solution:

- Each service performs its local transaction
- If any step fails, execute compensating transactions
- Maintains data consistency across services

Part 2: How You Implemented SAGA in Your Project

Answer: YES! You ARE using SAGA pattern with Outbox Pattern.

Your SAGA Implementation Components

1. *OutboxMessage Entity*

File: Ordering.Core/Entities/OutboxMessage.cs

```
public class OutboxMessage : EntityBase
{
    public string? Type { get; set; }           // Event type
    public string? Content { get; set; }        // Serialized event
    public string? CorrelationId { get; set; }   // Track flow
    public DateTime OccuredOn { get; set; }      // When created
    public DateTime? ProcessedOn { get; set; }   // When published
    public bool IsProcessed => ProcessedOn.HasValue;
    public string? Error { get; set; }          // Error if failed
}
```

2. *CreateOrderHandler with SAGA*

File: Ordering.Application/Orders/CreateOrder/CreateOrderHandler.cs (Lines 21-24)

```
public async Task<int> Handle(CreateOrderCommand command,
Cancellation token cancellationToken)
{
    var orderEntity = command.ToEntity();
    var generatedOrder = await _orderRepository.AddAsync(orderEntity);

    // SAGA starts here - Create outbox message
```

```

        var outboxMessage = OrderMapper.ToOutboxMessage(generatedOrder,
command.CorrelationId);

        // Save to database so dispatcher can pick it up
        await _orderRepository.AddOutboxMessageAsync(outboxMessage);

        return generatedOrder.Id;
    }

```

3. *OutboxMessageDispatcher (SAGA Orchestrator)*

File: Ordering.Infrastructure/Dispatcher/OutboxMessageDispatcher.cs

```

public class OutboxMessageDispatcher : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken
stoppingToken)
    {
        while(!stoppingToken.IsCancellationRequested)
        {
            var pendingMessage = await dbContext.OutboxMessage
                .Where(x => x.ProcessedOn == null)
                .OrderBy(x => x.OccuredOn)
                .Take(20)
                .ToListAsync();

            foreach (var message in pendingMessage)
            {
                try
                {
                    var orderCreatedEvent =
JsonConvert.DeserializeObject<OrderCreatedEvent>(message.Content);
                    await publishEndpoint.Publish(orderCreatedEvent);
// SAGA step
                    message.ProcessedOn = DateTime.UtcNow;
                }
                catch(Exception ex)
                {

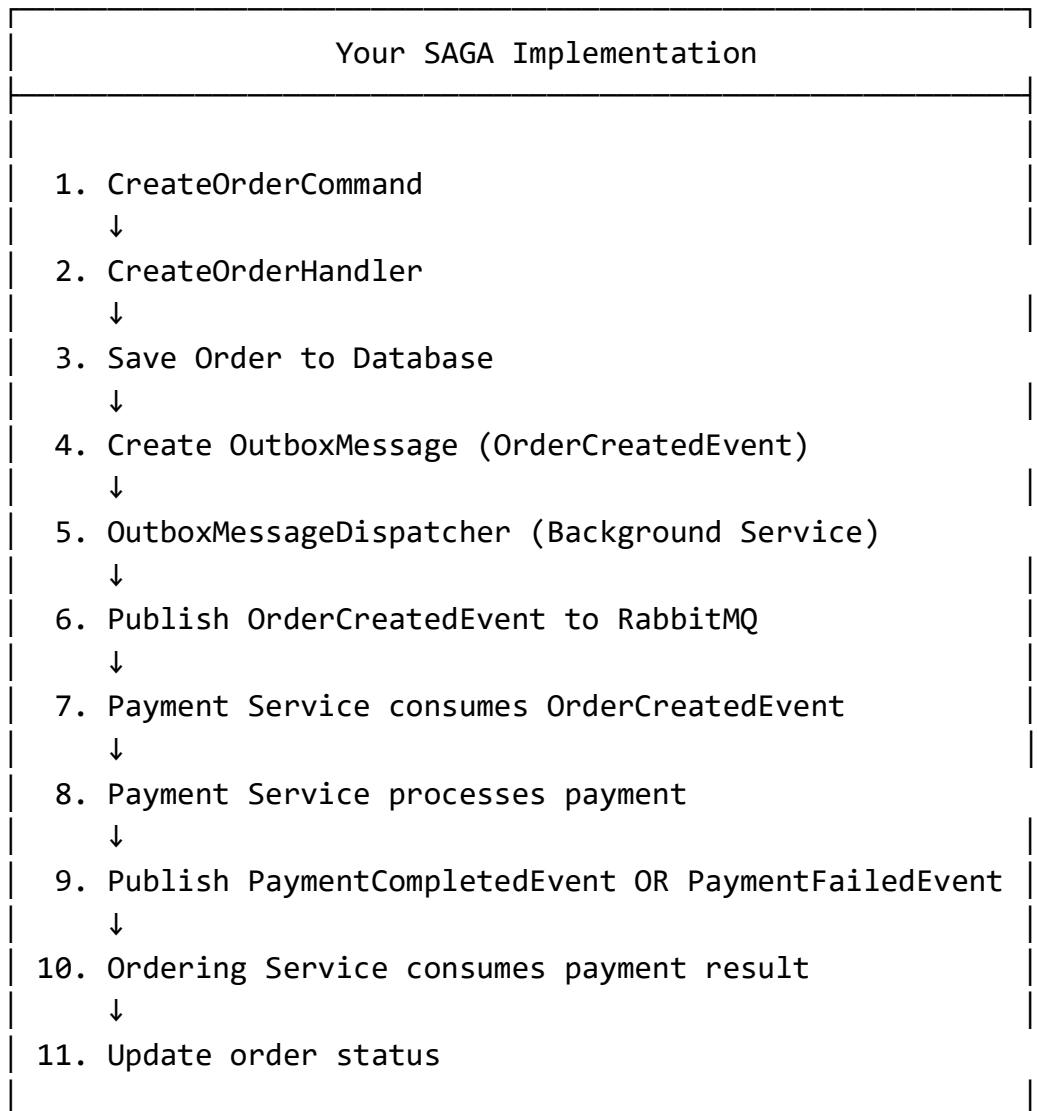
```

```

        message.Error = ex.Message; // SAGA failure
    }
}
await dbContext.SaveChangesAsync(stoppingToken);
await Task.Delay(10000, stoppingToken);
}
}
}

```

Your SAGA Flow



Part 3: What Problems Your SAGA Solves

Problem 1: Distributed Transaction Management

Without SAGA:

Basket Service → Delete Basket
Ordering Service → Create Order
Payment Service → Process Payment

- ✗ If Payment fails:
- Basket deleted
 - Order created
 - Payment failed
 - INCONSISTENT STATE!

With Your SAGA:

Basket Service → Delete Basket (can't be undone)
Ordering Service → Create Order + OutboxMessage
OutboxDispatcher → Publish OrderCreatedEvent
Payment Service → Process Payment
Payment Service → Publish PaymentCompletedEvent/FailedEvent
Ordering Service → Update order status

- ✓ If Payment fails:
- Order status = Failed
 - Can implement compensation logic
 - Consistent state maintained

Problem 2: Message Reliability

Your Outbox Pattern Solution:

Order Created → Save to OutboxTable (Database)



Background Service reads OutboxTable



Publish to RabbitMQ



Mark as Processed

✅ Benefits:

- No message loss if RabbitMQ is down
- Atomic operation (Order + OutboxMessage in same transaction)
- Retry mechanism built-in

Problem 3: Event Ordering

Your Implementation:

```
var pendingMessage = await dbContext.OutboxMessage
    .Where(x => x.ProcessedOn == null)
    .OrderBy(x => x.OccuredOn) // Process in order
    .Take(20)
    .ToListAsync();
```

Ensures events are processed in the order they were created.

Part 4: Step-by-Step SAGA Implementation Guide

Step 1: Create OutboxMessage Entity

```
public class OutboxMessage : EntityBase
{
    public string Type { get; set; } // Event type
    public string Content { get; set; } // Serialized
    event
    public string CorrelationId { get; set; } // Track flow
    public DateTime OccuredOn { get; set; } // When
    created
```

```

        public DateTime? ProcessedOn { get; set; }           // When
published
        public bool IsProcessed => ProcessedOn.HasValue;
        public string Error { get; set; }                 // Error if
failed
    }

```

Step 2: Add OutboxMessage to DbContext

```

public class OrderContext : DbContext
{
    public DbSet<OutboxMessage> OutboxMessages { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<OutboxMessage>(entity =>
        {
            entity.HasKey(e => e.Id);
            entity.Property(e => e.Type).IsRequired();
            entity.Property(e => e.Content).IsRequired();
            entity.Property(e => e.CorrelationId).IsRequired();
            entity.Property(e => e.OccuredOn).IsRequired();
            entity.Property(e => e.ProcessedOn);
            entity.Property(e => e.Error);
        });
    }
}

```

Step 3: Create OutboxMessage Constants

```

public static class OutboxMessageTypes
{
    public const string OrderCreated = "OrderCreated";
    public const string OrderCancelled = "OrderCancelled";
    public const string PaymentCompleted = "PaymentCompleted";
    public const string PaymentFailed = "PaymentFailed";
}

```

```
}
```

Step 4: Create OutboxMessage Mapper

```
public static class OrderMapper
{
    public static OutboxMessage ToOutboxMessage(Order order, Guid
correlationId)
    {
        return new OutboxMessage
        {
            CorrelationId = correlationId.ToString(),
            Type = OutboxMessageTypes.OrderCreated,
            OccuredOn = DateTime.UtcNow,
            Content = JsonConvert.SerializeObject(new
OrderCreatedEvent
            {
                Id = order.Id,
                UserName = order.UserName,
                TotalPrice = order.TotalPrice,
                // ... other properties
                CorrelationId = correlationId
            })
        };
    }
}
```

Step 5: Update Repository to Handle OutboxMessages

```
public interface IOrderRepository : IAsyncRepository<Order>
{
    Task AddOutboxMessageAsync(OutboxMessage outboxMessage);
    Task<IEnumerable<OutboxMessage>> GetPendingOutboxMessages();
    Task MarkOutboxMessageAsProcessed(int messageId);
}

public class OrderRepository : RepositoryBase<Order>, IOrderRepository
```

```

{
    public async Task AddOutboxMessageAsync(OutboxMessage
outboxMessage)
    {
        await _dbContext.OutboxMessages.AddAsync(outboxMessage);
    }

    public async Task<IEnumerable<OutboxMessage>>
GetPendingOutboxMessages()
    {
        return await _dbContext.OutboxMessages
            .Where(x => x.ProcessedOn == null)
            .OrderBy(x => x.OccuredOn)
            .ToListAsync();
    }

    public async Task MarkOutboxMessageAsProcessed(int messageId)
    {
        var message = await
_dbContext.OutboxMessages.FindAsync(messageId);
        if (message != null)
        {
            message.ProcessedOn = DateTime.UtcNow;
            await _dbContext.SaveChangesAsync();
        }
    }
}

```

Step 6: Create OutboxMessageDispatcher

```

public class OutboxMessageDispatcher : BackgroundService
{
    private readonly IServiceProvider _serviceProvider;
    private readonly ILogger<OutboxMessageDispatcher> _logger;

    public OutboxMessageDispatcher(IServiceProvider serviceProvider,
ILogger<OutboxMessageDispatcher> logger)
    {

```

```

        _serviceProvider = serviceProvider;
        _logger = logger;
    }

    protected override async Task ExecuteAsync(CancellationToken
stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            using var scope = _serviceProvider.CreateScope();
            var orderRepository =
scope.ServiceProvider.GetRequiredService<IOrderRepository>();
            var publishEndpoint =
scope.ServiceProvider.GetRequiredService<IPublishEndpoint>();

            var pendingMessages = await
orderRepository.GetPendingOutboxMessages();

            foreach (var message in pendingMessages)
            {
                try
                {
                    switch (message.Type)
                    {
                        case OutboxMessageTypes.OrderCreated:
                            var orderCreatedEvent =
JsonConvert.DeserializeObject<OrderCreatedEvent>(message.Content);
                            await
publishEndpoint.Publish(orderCreatedEvent);
                            break;

                        case OutboxMessageTypes.OrderCancelled:
                            var orderCancelledEvent =
JsonConvert.DeserializeObject<OrderCancelledEvent>(message.Content);
                            await
publishEndpoint.Publish(orderCancelledEvent);
                            break;
                    }
                }
            }
        }
    }

```

```

        await
orderRepository.MarkOutboxMessageAsProcessed(message.Id);
        _logger.LogInformation("Published outbox message
{Id}", message.Id);
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Failed to publish outbox
message {Id}", message.Id);
        // Could implement retry logic here
    }
}

await Task.Delay(10000, stoppingToken); // Check every 10
seconds
    }
}
}

```

Step 7: Register OutboxMessageDispatcher

```

// In Program.cs
builder.Services.AddHostedService<OutboxMessageDispatcher>();

```

Step 8: Update Command Handlers to Use SAGA

```

public class CreateOrderHandler : ICommandHandler<CreateOrderCommand,
int>
{
    private readonly IOrderRepository _orderRepository;
    private readonly ILogger<CreateOrderHandler> _logger;

    public async Task<int> Handle(CreateOrderCommand command,
CancellationTokens cancellationToken)
    {
        // Step 1: Create order (local transaction)
        var orderEntity = command.ToEntity();
    }
}

```

```

        var generatedOrder = await
_orderRepository.AddAsync(orderEntity);

        // Step 2: Create outbox message (part of same transaction)
        var outboxMessage =
OrderMapper.ToOutboxMessage(generatedOrder, command.CorrelationId);
        await _orderRepository.AddOutboxMessageAsync(outboxMessage);

        _logger.LogInformation($"Order with ID {generatedOrder.Id}
successfully created with outbox message");
        return generatedOrder.Id;
    }
}

```

Part 5: Adding Compensation Logic

Step 1: Create Compensation Events

```

public class OrderCancelledEvent : BaseIntegrationEvent
{
    public int OrderId { get; set; }
    public string Username { get; set; }
    public string Reason { get; set; }
    public Guid CorrelationId { get; set; }
}

```

Step 2: Create Compensation Handler

```

public class PaymentFailedConsumer : IConsumer<PaymentFailedEvent>
{
    private readonly IOrderRepository _orderRepository;
    private readonly IPublishEndpoint _publishEndpoint;
    private readonly ILogger<PaymentFailedConsumer> _logger;

    public async Task Consume(ConsumeContext<PaymentFailedEvent>
context)
    {

```

```

var message = context.Message;

// Step 1: Update order status
var order = await
_orderRepository.GetByIdAsync(message.OrderId);
if (order != null)
{
    order.Status = OrderStatus.Failed;
    await _orderRepository.UpdateAsync(order);

    // Step 2: Create compensation event
    var compensationEvent = new OrderCancelledEvent
    {
        OrderId = message.OrderId,
        UserName = order.UserName,
        Reason = message.Reason,
        CorrelationId = message.CorrelationId
    };

    // Step 3: Add to outbox for reliable publishing
    var outboxMessage = new OutboxMessage
    {
        Type = OutboxMessageTypes.OrderCancelled,
        Content =
            JsonConvert.SerializeObject(compensationEvent),
        CorrelationId = message.CorrelationId.ToString(),
        OccuredOn = DateTime.UtcNow
    };

    await
_orderRepository.AddOutboxMessageAsync(outboxMessage);

    _logger.LogInformation("Order {OrderId} cancelled due to
payment failure", message.OrderId);
}
}
}

```

Part 6: Interview Questions & Answers for SAGA Pattern

Basic Questions

Q1: What is the SAGA pattern?

A: SAGA is a sequence of local transactions where each transaction performs an action and publishes an event. If any transaction fails, the saga executes compensating transactions to undo previous actions. It manages distributed transactions across microservices.

Q2: Why do we need SAGA instead of traditional transactions?

A: Traditional transactions (ACID) don't work across microservices because:

- Can't span multiple databases/services
- Long-running transactions block resources
- Network failures cause complete rollbacks
- Need for eventual consistency

Q3: What are the two types of SAGA patterns?

A:

1. **Choreography:** Each service publishes events and reacts to events from others (decentralized)
2. **Orchestration:** A central coordinator tells each service what to do (centralized)

Your Project-Specific Questions

Q4: How did you implement SAGA in your project?

A: I implemented SAGA using the **Outbox Pattern**:

- Created OutboxMessage entity to store events in database
- CreateOrderHandler saves order + creates outbox message atomically
- OutboxMessageDispatcher background service publishes events to RabbitMQ
- Services consume events and can publish compensation events

Q5: What is the Outbox Pattern and why did you use it?

A: The Outbox Pattern ensures reliable event publishing:

- Events are stored in database as part of the same transaction
- Background service reads from outbox table and publishes to message broker
- Guarantees no message loss if message broker is down
- Provides atomicity between business operation and event publishing

Q6: Show me your SAGA flow in your ECommerce project

A:

1. CreateOrderCommand → CreateOrderHandler
2. Save Order to Database + Create OutboxMessage
3. OutboxMessageDispatcher publishes OrderCreatedEvent
4. Payment Service consumes OrderCreatedEvent
5. Payment Service processes payment
6. Payment Service publishes PaymentCompletedEvent OR PaymentFailedEvent
7. Ordering Service consumes payment result
8. If payment failed → Update order status + publish OrderCancelledEvent

Advanced Questions

Q7: How do you handle compensation in your SAGA?

A:

- Payment service publishes PaymentFailedEvent on failure
- Ordering service consumes PaymentFailedEvent
- Updates order status to "Failed"
- Can publish OrderCancelledEvent for other services to compensate
- Each service implements its own compensation logic

Q8: What happens if your OutboxMessageDispatcher fails?

A:

- Messages remain in ProcessedException = null state

- Dispatcher retries every 10 seconds
- Failed messages have error information logged
- Can implement retry logic with exponential backoff
- Eventually consistent delivery

Q9: How do you ensure event ordering in SAGA?

A:

```
var pendingMessage = await dbContext.OutboxMessage
    .Where(x => x.ProcessedOn == null)
    .OrderBy(x => x.OccuredOn) // Process in
creation order
    .ToListAsync();
```

- Events are processed in the order they were created
- OccuredOn timestamp ensures FIFO processing
- Critical for maintaining business logic sequence

Scenario Questions

Q10: Your payment service fails after processing payment but before publishing the event. What happens?

A:

- Payment is processed (money deducted)
- Event not published → Other services don't know about payment
- Need to implement idempotency in payment service
- Payment service should be able to retry publishing the event
- Could implement a separate outbox table in payment service

Q11: How would you add inventory management to your SAGA?

A:

1. Create InventoryUpdatedEvent and InventoryUpdateFailedEvent
2. Add inventory service consumer for OrderCreatedEvent
3. Inventory service updates stock and publishes result

4. If inventory fails → publish `OrderCancelledEvent`
5. Ordering service updates order status and handles compensation

Q12: What about long-running SAGAs that take hours to complete?

A:

- Use timeout handling in each service
- Implement saga timeout events
- Store saga state in database
- Use correlation ID to track long-running processes
- Consider adding monitoring and alerting for stuck sagas

Implementation Questions

Q13: How do you test your SAGA implementation?

A:

- Unit tests for individual handlers
- Integration tests for outbox dispatcher
- Contract tests for event publishing/consuming
- End-to-end tests with test containers
- Mock message broker for unit tests

Q14: How do you monitor your SAGA?

A:

- Log correlation IDs throughout the flow
- Monitor outbox message queue length
- Track processing times and failures
- Add health checks for background services
- Use distributed tracing (Jaeger, Zipkin)

Comparison Questions

Q15: What's the difference between your implementation and 2PC (Two-Phase Commit)?

A:

- **2PC:** Synchronous, blocking, all-or-nothing, doesn't scale
- **SAGA:** Asynchronous, non-blocking, eventual consistency, scales well
- **2PC:** Requires distributed transaction coordinator
- **SAGA:** Each service manages its own transactions

Q16: Why did you choose Choreography over Orchestration?

A:

- **Choreography:** Decoupled services, no single point of failure, more scalable
- **Orchestration:** Centralized control, easier to manage complex flows, single point of failure
- **My choice:** Choreography with outbox pattern for reliability and scalability

Common Pitfalls Questions

Q17: What are the common pitfalls in SAGA implementation?

A:

- Not handling compensation properly
- Message ordering issues
- Duplicate event processing (need idempotency)
- Long-running transactions without timeout
- Not monitoring saga failures
- Inconsistent event schemas

Q18: How do you ensure idempotency in your consumers?

A:

- Check if event already processed using correlation ID
- Use database constraints to prevent duplicate operations
- Implement deduplication logic in consumers
- Store processed event IDs for reference

Performance Questions

Q19: How does your Outbox Pattern affect performance?

A:

- Additional database writes for outbox messages
- Background service adds overhead
- Trade-off: Reliability vs performance
- Can optimize with batch processing
- Consider using separate database for outbox table

Q20: How do you scale your SAGA implementation?

A:

- Scale outbox dispatcher horizontally
- Partition outbox table by service
- Use message broker partitioning
- Implement circuit breakers for external calls
- Add caching for frequently accessed data

Summary

I've provided a comprehensive guide covering:

1. SAGA Pattern in Your Project

-  **You ARE using SAGA** with Outbox Pattern
- **Components:** OutboxMessage entity, OutboxMessageDispatcher, event publishing
- **Flow:** Create order → Outbox message → Background dispatcher → RabbitMQ → Payment service

2. Why SAGA is Needed

- **Distributed transactions** across microservices
- **Eventual consistency** instead of ACID
- **Compensation logic** for failed operations

- **Message reliability** with Outbox Pattern

3. Your Implementation Details

- **OutboxMessage**: Stores events in database (atomic with business transaction)
- **OutboxMessageDispatcher**: Background service publishes events reliably
- **Correlation ID**: Tracks request flow across services
- **Event ordering**: Processes events in creation order

4. Step-by-Step Implementation Guide

- Create OutboxMessage entity
- Add to DbContext
- Create dispatcher background service
- Update command handlers to use outbox
- Add compensation logic

5. Interview Preparation (20+ Questions)

- **Basic**: What is SAGA, why need it, types of SAGA
- **Project-specific**: Your implementation, Outbox Pattern, event flow
- **Advanced**: Compensation, failure handling, monitoring
- **Scenario**: Payment failures, inventory management, long-running sagas
- **Implementation**: Testing, scaling, performance considerations

Key Takeaways for Interviews

Your SAGA Implementation:

- Uses **Choreography pattern** (decentralized)
- **Outbox Pattern** for reliable event publishing
- **Event-driven** communication via RabbitMQ
- **Compensation** through failed events
- **Correlation ID** for request tracing

Benefits:

-  No message loss (outbox pattern)

-  Atomic operations (order + event in same transaction)
-  Scalable architecture
-  Eventual consistency

This comprehensive guide should prepare you for any SAGA-related interview questions and help you explain your project's distributed transaction management effectively.

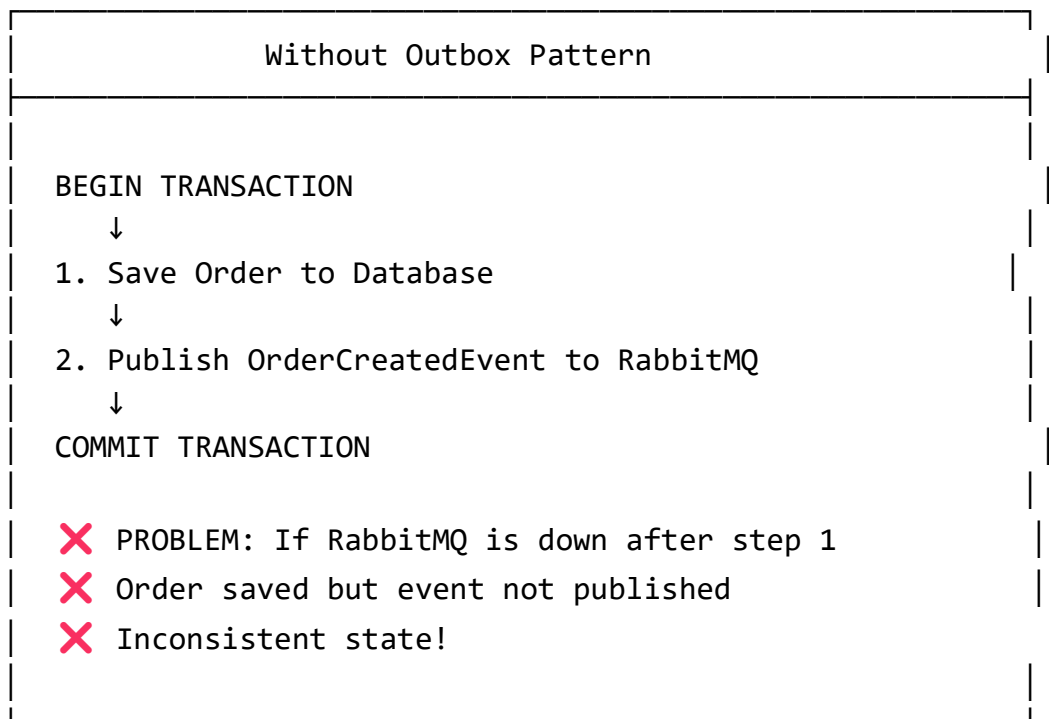
Outbox Pattern + SAGA - Complete Guide

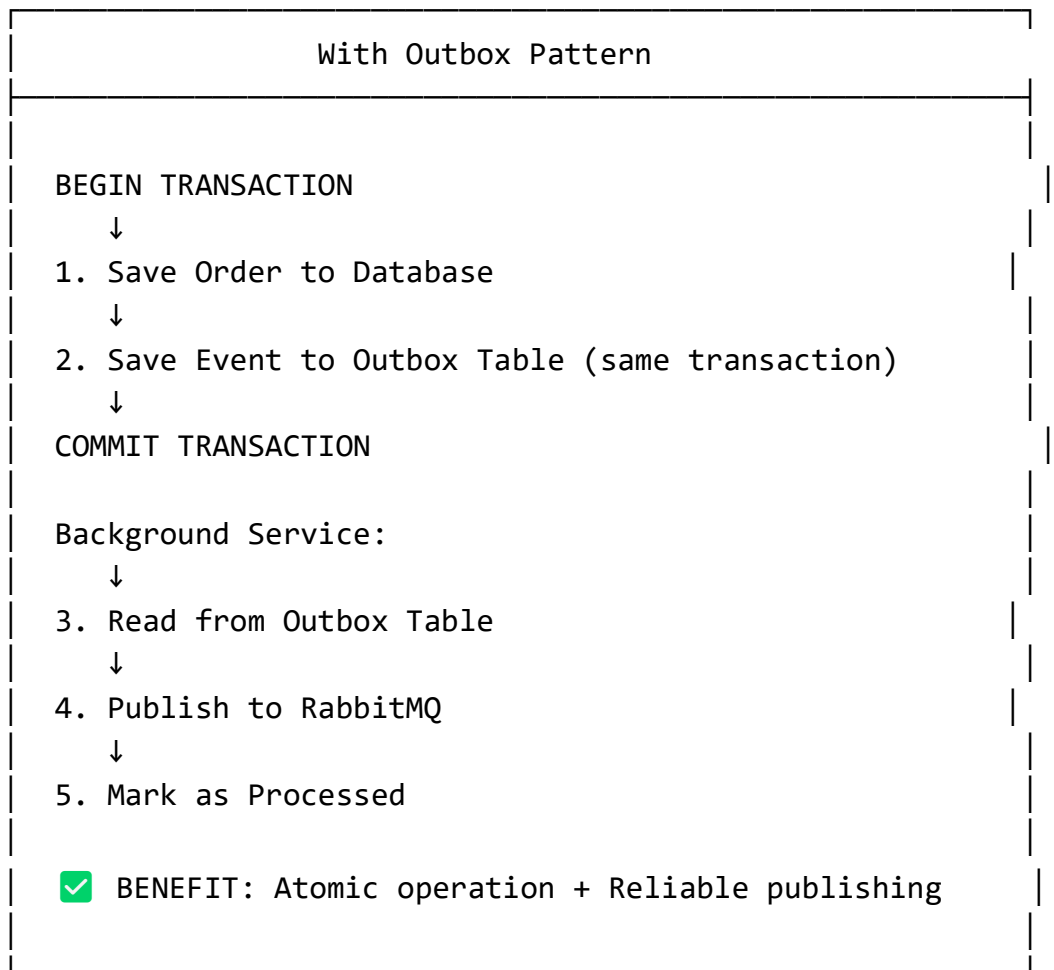
Using Your Project

Part 1: What is Outbox Pattern?

Definition

The Outbox Pattern is a **reliable event publishing mechanism** that ensures events are not lost when publishing to a message broker. It stores events in a database table (outbox) as part of the same transaction, then a background service publishes them to the message broker.





Part 2: Why Outbox Pattern is Needed

The Problem: Event Publishing Reliability

Scenario: Your ECommerce checkout process

User clicks "Checkout"

↓

Ordering Service:

1. Create Order (Database)
2. Publish OrderCreatedEvent (RabbitMQ)

↓

Payment Service receives OrderCreatedEvent

↓

Payment Service processes payment

What happens if RabbitMQ is down?

Ordering Service:

1. Create Order  (Saved to database)
2. Publish OrderCreatedEvent  (RabbitMQ down)

Result:

- Order exists in database
- Payment service never gets the event
- User thinks order succeeded but payment never processed
- INCONSISTENT STATE!

The Outbox Solution

Ordering Service:

1. Create Order 
2. Save OrderCreatedEvent to Outbox Table  (same transaction)

Background Service:

3. Read OrderCreatedEvent from Outbox Table
4. Publish to RabbitMQ
5. Mark as Processed

Result:

- Atomic operation: Both order and event saved together
- If RabbitMQ is down, event stays in outbox table
- Background service retries until successful
- CONSISTENT STATE!

Part 3: How You Implemented Outbox Pattern in Your Project

Your Outbox Implementation Components

1. *OutboxMessage Entity*

File: Ordering.Core/Entities/OutboxMessage.cs

```
public class OutboxMessage : EntityBase
{
    public string? Type { get; set; }           // Event type
    public string? Content { get; set; }        // Serialized event
    public string? CorrelationId { get; set; }  // Track request flow
    public DateTime OccuredOn { get; set; }     // When created
    public DateTime? ProcessedOn { get; set; }  // When published
    public bool IsProcessed => ProcessedOn.HasValue;
    public string? Error { get; set; }         // Error if failed
}
```

Key Fields Explained:

- **Type:** Event type for routing (OrderCreated, OrderCancelled)
- **Content:** Serialized JSON event data
- **CorrelationId:** Links all events in a single flow
- **ProcessedOn:** When successfully published
- **Error:** Failure information for retry logic

2. *Database Integration*

OutboxMessage is part of your OrderContext (from migrations):

```
CREATE TABLE [OutboxMessage] (
    [Id] int NOT NULL PRIMARY KEY,
    [Type] nvarchar(max) NOT NULL,
    [Content] nvarchar(max) NOT NULL,
    [CorrelationId] nvarchar(max) NOT NULL,
```

```
[OccuredOn] datetime2 NOT NULL,  
[ProcessedOn] datetime2 NULL,  
[Error] nvarchar(max) NULL  
);
```

3. CreateOrderHandler with Outbox

File: Ordering.Application/Orders/CreateOrder/CreateOrderHandler.cs (Lines 21-24)

```
public async Task<int> Handle(CreateOrderCommand command,  
Cancellation token cancellationToken)  
{  
    // Step 1: Create order (business operation)  
    var orderEntity = command.ToEntity();  
    var generatedOrder = await _orderRepository.AddAsync(orderEntity);  
  
    // Step 2: Create outbox message (same transaction)  
    var outboxMessage = OrderMapper.ToOutboxMessage(generatedOrder,  
command.CorrelationId);  
    await _orderRepository.AddOutboxMessageAsync(outboxMessage);  
  
    // Both operations are in the same transaction - atomic!  
    return generatedOrder.Id;  
}
```

Key Point: Order creation and outbox message creation are in the same database transaction.

4. OutboxMessageMapper

File: Ordering.Application/Mappers/OrderMapper.cs (Lines 121-147)

```
public static OutboxMessage ToOutboxMessage(Order order, Guid  
correlationId)  
{  
    return new OutboxMessage  
    {  
        CorrelationId = correlationId.ToString(),
```

```

        Type = OutboxMessageTypes.OrderCreated,
        OccuredOn = DateTime.UtcNow,
        Content = JsonConvert.SerializeObject(new
        {
            order.Id,
            order.UserName,
            order.TotalPrice,
            order.FirstName,
            order.LastName,
            order.AddressLine,
            order.Country,
            order.State,
            order.ZipCode,
            order.CardName,
            order.CardNumber,
            order.Expiration,
            order.Cvv,
            order.PaymentMethod,
            order.Status
        })
    };
}

```

5. *OutboxMessageDispatcher (Background Service)*

File: Ordering.Infrastructure/Dispatcher/OutboxMessageDispatcher.cs

```

public class OutboxMessageDispatcher : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken
stoppingToken)
    {
        while(!stoppingToken.IsCancellationRequested)
        {
            using var scope = _serviceProvider.CreateScope();
            var dbContext =
scope.ServiceProvider.GetRequiredService<OrderContext>();
            var publishEndpoint =

```

```

scope.ServiceProvider.GetRequiredService<IPublishEndpoint>();

    // Step 1: Get unprocessed messages
    var pendingMessage = await dbContext.OutboxMessage
        .Where(x => x.ProcessedOn == null)
        .OrderBy(x => x.OccuredOn) //
Process in order

        .Take(20)
        .ToListAsync();

    // Step 2: Process each message
    foreach (var message in pendingMessage)
    {
        try
        {
            // Step 3: Deserialize and publish event
            var orderCreatedEvent =
JsonConvert.DeserializeObject<OrderCreatedEvent>(message.Content);
            await publishEndpoint.Publish(orderCreatedEvent);

            // Step 4: Mark as processed
            message.ProcessedOn = DateTime.UtcNow;
            _logger.LogInformation("Published outbox message
{Id}", message.Id);
        }
        catch(Exception ex)
        {
            // Step 5: Handle failure
            message.Error = ex.Message;
            _logger.LogError(ex,"Failed to publish outBox
Message {Id}",message.Id);
        }
    }

    // Step 6: Save changes
    await dbContext.SaveChangesAsync(stoppingToken);

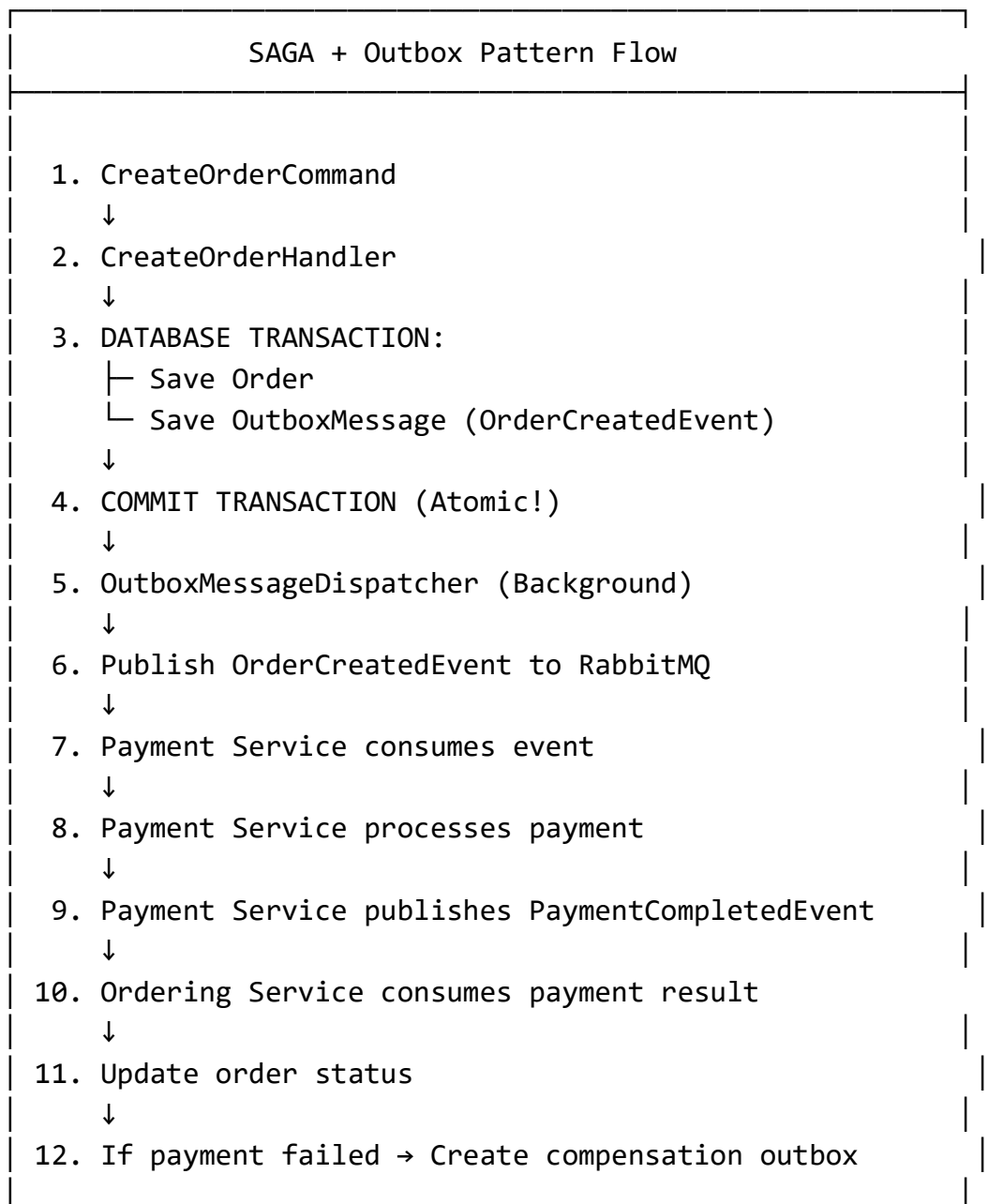
    // Step 7: Wait before next batch
    await Task.Delay(10000, stoppingToken);

```

```
}  
    }  
}
```

Part 4: How SAGA + Outbox Work Together

The Combined Pattern



Why Both Patterns Are Needed

SAGA without Outbox:

Order Service → Create Order → Publish Event (RabbitMQ)

Payment Service → Process Payment → Publish Event (RabbitMQ)

✖ Problems:

- If RabbitMQ down, events lost
- No guarantee events are published
- Inconsistent state

SAGA with Outbox:

Order Service → Create Order + Save to Outbox → Background publishes

Payment Service → Process Payment + Save to Outbox → Background publishes

✔ Benefits:

- Atomic operations (business + events)
- Reliable event publishing
- Event ordering guaranteed
- No message loss

Part 5: What Problems SAGA + Outbox Solve

Problem 1: Message Loss Prevention

Your Solution:

```
// Atomic transaction - both succeed or both fail
await _orderRepository.AddAsync(orderEntity);
await _orderRepository.AddOutboxMessageAsync(outboxMessage);
```

Problem 2: Event Ordering

Your Solution:

```
var pendingMessage = await dbContext.OutboxMessage
    .Where(x => x.ProcessedOn == null)
    .OrderBy(x => x.OccuredOn) // FIFO ordering
    .ToListAsync();
```

Problem 3: Transaction Boundaries

Your Solution:

- Business operation and event creation in same transaction
- Background service handles publishing outside transaction
- Clear separation of concerns

Problem 4: Retry Mechanism

Your Solution:

```
while(!stoppingToken.IsCancellationRequested)
{
    // Process messages
    await Task.Delay(10000, stoppingToken); // Retry every 10 seconds
}
```

Part 6: Step-by-Step Implementation Guide

Step 1: Create OutboxMessage Entity

```
public class OutboxMessage : EntityBase
{
    public string Type { get; set; }
    public string Content { get; set; }
    public string CorrelationId { get; set; }
    public DateTime OccuredOn { get; set; }
```

```

    public DateTime? ProcessedOn { get; set; }
    public bool IsProcessed => ProcessedOn.HasValue;
    public string Error { get; set; }
}

```

Step 2: Add to DbContext

```

public class OrderContext : DbContext
{
    public DbSet<OutboxMessage> OutboxMessages { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<OutboxMessage>(entity =>
        {
            entity.HasKey(e => e.Id);
            entity.Property(e => e.Type).IsRequired();
            entity.Property(e => e.Content).IsRequired();
        });
    }
}

```

Step 3: Create Repository Methods

```

public interface IOrderRepository : IAsyncRepository<Order>
{
    Task AddOutboxMessageAsync(OutboxMessage outboxMessage);
    Task<IEnumerable<OutboxMessage>> GetPendingOutboxMessages();
}

public class OrderRepository : RepositoryBase<Order>, IOrderRepository
{
    public async Task AddOutboxMessageAsync(OutboxMessage
outboxMessage)
    {
        await _dbContext.OutboxMessages.AddAsync(outboxMessage);
    }
}

```

```

        public async Task<IEnumerable<OutboxMessage>>
GetPendingOutboxMessages()
    {
        return await _dbContext.OutboxMessages
            .Where(x => x.ProcessedOn == null)
            .OrderBy(x => x.OccuredOn)
            .ToListAsync();
    }
}

```

Step 4: Update Command Handlers

```

public class CreateOrderHandler : ICommandHandler<CreateOrderCommand,
int>
{
    public async Task<int> Handle(CreateOrderCommand command,
CancellationTokens cancellationTokens)
    {
        // Business operation
        var orderEntity = command.ToEntity();
        var generatedOrder = await
_orderRepository.AddAsync(orderEntity);

        // Event creation (same transaction)
        var outboxMessage =
OrderMapper.ToOutboxMessage(generatedOrder, command.CorrelationId);
        await _orderRepository.AddOutboxMessageAsync(outboxMessage);

        return generatedOrder.Id;
    }
}

```

Step 5: Create Background Dispatcher

```

public class OutboxMessageDispatcher : BackgroundService
{

```

```

private readonly IServiceProvider _serviceProvider;
private readonly ILogger<OutboxMessageDispatcher> _logger;

protected override async Task ExecuteAsync(CancellationTok
stoppingToken)
{
    while (!stoppingToken.IsCancellationRequested)
    {
        using var scope = _serviceProvider.CreateScope();
        var repository =
scope.ServiceProvider.GetRequiredService<IOrderRepository>();
        var publishEndpoint =
scope.ServiceProvider.GetRequiredService<IPublishEndpoint>();

        var messages = await
repository.GetPendingOutboxMessages();

        foreach (var message in messages)
        {
            try
            {
                var eventData =
JsonConvert.DeserializeObject(message.Content);
                await publishEndpoint.Publish(eventData);
                message.ProcessedOn = DateTime.UtcNow;
            }
            catch (Exception ex)
            {
                message.Error = ex.Message;
                _logger.LogError(ex, "Failed to publish message
{Id}", message.Id);
            }
        }

        await Task.Delay(10000, stoppingToken);
    }
}

```

Step 6: Register Services

```
// In Program.cs
builder.Services.AddHostedService<OutboxMessageDispatcher>();
```

Part 7: Advanced Features You Can Add

1. Retry Logic with Exponential Backoff

```
private async Task<bool> PublishWithRetry(OutboxMessage message,
IPublishEndpoint publishEndpoint, int retryCount = 3)
{
    for (int i = 0; i < retryCount; i++)
    {
        try
        {
            var eventData =
JsonConvert.DeserializeObject(message.Content);
            await publishEndpoint.Publish(eventData);
            return true;
        }
        catch (Exception ex)
        {
            if (i == retryCount - 1)
            {
                message.Error = ex.Message;
                return false;
            }
            await Task.Delay(TimeSpan.FromSeconds(Math.Pow(2, i)));
        }
    }
    return false;
}
```

2. Batch Processing

```
// Process multiple messages at once
var pendingMessages = await repository.GetPendingOutboxMessages();
var eventsToPublish = new List<object>();

foreach (var message in pendingMessages)
{

eventsToPublish.Add(JsonConvert.DeserializeObject(message.Content));
}

// Publish all at once (if message broker supports batch)
await publishEndpoint.PublishBatch(eventsToPublish);
```

3. Dead Letter Queue

```
// After N retries, move to dead letter table
if (message.RetryCount >= 3)
{
    var deadLetterMessage = new DeadLetterMessage
    {
        OriginalMessage = message,
        RetryCount = message.RetryCount,
        LastError = message.Error,
        CreatedAt = DateTime.UtcNow
    };
    await repository.AddToDeadLetterAsync(deadLetterMessage);
    await repository.DeleteOutboxMessageAsync(message.Id);
}
```

Part 8: Monitoring and Observability

1. Outbox Queue Length Monitoring

```
// Add health check
public class OutboxHealthCheck : IHealthCheck
```

```

{
    public async Task<HealthCheckResult>
    CheckHealthAsync(HealthCheckContext context)
    {
        var pendingCount = await
        _repository.GetPendingOutboxMessageCount();

        if (pendingCount > 1000)
        {
            return HealthCheckResult.Unhealthy($"Too many pending
messages: {pendingCount}");
        }

        return HealthCheckResult.Healthy($"Pending messages:
{pendingCount}");
    }
}

```

2. Processing Time Metrics

```

// Track processing time
var stopwatch = Stopwatch.StartNew();
await publishEndpoint.Publish(eventData);
stopwatch.Stop();

_logger.LogInformation("Message {MessageId} processed in
{ElapsedMs}ms",
    message.Id, stopwatch.ElapsedMilliseconds);

```

Part 9: Common Pitfalls and Solutions

Pitfall 1: Large Event Payloads

Problem: Events become too large, affecting performance

Solution:

```
// Store only essential data in events
public class OrderCreatedEvent
{
    public int OrderId { get; set; }
    public string UserName { get; set; }
    public decimal TotalPrice { get; set; }
    // Don't include all order details
}
```

Pitfall 2: Outbox Table Growth

Problem: Outbox table grows indefinitely

Solution:

```
// Clean up processed messages
await repository.DeleteProcessedOutboxMessagesAsync(olderThan:
DateTime.UtcNow.AddDays(7));
```

Pitfall 3: Duplicate Processing

Problem: Same event processed multiple times

Solution:

```
// Add unique constraint on message type + correlation ID
modelBuilder.Entity<OutboxMessage>()
    .HasIndex(e => new { e.Type, e.CorrelationId })
    .IsUnique();
```

Summary

Your Implementation:

-  **Outbox Pattern:** Reliable event publishing
-  **SAGA Pattern:** Distributed transaction management
-  **Combined:** Atomic business operations + reliable events

Key Benefits:

- **No message loss:** Events stored in database before publishing
- **Event ordering:** FIFO processing guaranteed
- **Atomic operations:** Business + events in same transaction
- **Retry mechanism:** Background service retries failed publishes
- **Monitoring:** Track processing status and errors

Your Current Flow:

CreateOrderCommand → Save Order + OutboxMessage → Background
Dispatcher → RabbitMQ → Payment Service

This combination ensures your microservices maintain data consistency while handling distributed transactions reliably.